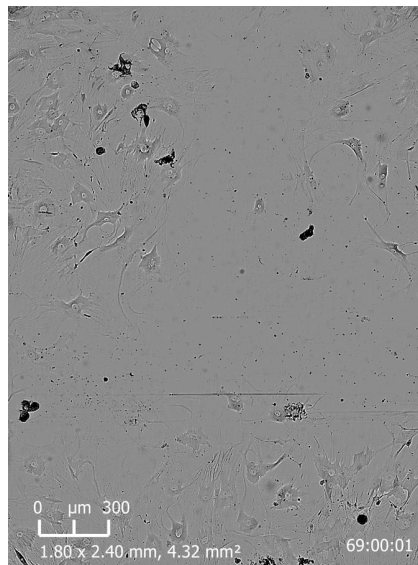


OPALS @ UC San Diego

Automated Cell Segmentation & Tracking

Teaching high-schoolers machine learning, data, and HPC through real microscopy



Example data: confluent fibroblasts (brightfield, $t = 69$ h) — weak boundaries, debris, and a plate scratch the commercial tool miscounts as cells.

4-week summer intensive + optional follow-up weeks 5–8

Mentor: Nathan · IEM OPALS High School Internship

Prepared May 2026

Contents

1. Program overview & goals

OPALS is UC San Diego's IEM high-school research internship. The summer track is an in-person, full-day, four-week experience that ends with each intern giving a research talk and/or poster. This plan turns that month into a single, shared engineering-and-science project that a small team builds together — while learning Python, machine learning, scientific data work, high-performance computing (HPC), and visualization, all grounded in real cell biology.

The scientific problem

Our lab images living cells over time (the attached example is a near-confluent fibroblast monolayer at 69 hours). To get numbers out — how many cells, how big, how fast they move and divide — the images must be segmented (each cell outlined) and tracked (followed frame to frame). The popular commercial package we currently rely on does this with classic computer-vision tools that:

- need constant manual tuning of thresholds and parameters for every new experiment;
- struggle with weak / translucent cell boundaries, merging neighboring cells together;
- mistake plate scratches and debris for cells (visible as the faint horizontal line in our example);
- run slowly, and only on a single licensed machine that is hard for the team to access.

What we will build

A more automated, more accurate, and portable replacement — a deep-learning pipeline (PyTorch, building on modern generalist segmentation models) that you point at a folder of images and run with a single command to get clean cell outlines, tracks, quantitative measurements, and figures. Because we now have an allocation on the San Diego Supercomputer Center (SDSC) through NSF ACCESS — our first time using it — the pipeline will run on real GPU hardware, processing entire experiments far faster than the single licensed machine.

The product goal in one sentence: import everything, press run, get the best results — no per-image babysitting, no single-seat license.

Learning goals

By the symposium, every intern — regardless of starting level — should be able to:

- read and write Python well enough to load, process, and visualize image data;
- explain what machine learning is, what training data and a loss function are, and why a learned model beats hand-tuned thresholds for weak-boundary cells;
- run and fine-tune a real PyTorch model and evaluate it honestly against ground truth;
- log in to a supercomputer, submit a job, and understand why HPC matters for science;
- turn results into clear scientific figures and tell the story of the project to a general audience.

2. The team & roles

The cohort is a small team (~2–5 interns) of mixed experience building one shared tool. Everyone touches every part during lessons, but each intern owns one component as its lead — this gives them a clear piece to present in the final talk while keeping the project integrated. With fewer than four interns, combine adjacent roles (e.g. Data + Visualization, Model + HPC).

Role	Owns	Final-talk story
Data Lead	Organizing the image dataset, hand-annotation / ground truth, train-val-test splits, data augmentation.	“How we taught the computer what a cell looks like.”
Model Lead	The PyTorch model: running Cellpose-SAM, fine-tuning, the training loop, accuracy metrics.	“How the model learns to outline cells.”
HPC / Infra Lead	SDSC accounts, environment setup, Slurm batch jobs, reproducibility, speed benchmarks.	“How we ran it on a supercomputer.”
Visualization / Comms Lead	Figures, tracking overlay videos, the results dashboard, poster & talk design.	“What the cells actually did, and why it matters.”

Working with a mixed-level cohort

- **Pair programming:** pair a more-experienced intern with a newer one; rotate pairs weekly so knowledge spreads.
- **Core + stretch:** every lab has a required “core” task everyone completes, plus optional “stretch” challenges for those moving fast (e.g. add an extra metric, try a second model, optimize a Slurm script).
- **Everyone ships:** each intern commits code to the shared GitHub repo every day, however small — this is how they learn version control and feel ownership.
- **Lab notebook:** each intern keeps a daily research notebook (a shared doc or markdown file) — what they tried, what worked, one question. This becomes raw material for the poster.

3. Technical approach & stack

We deliberately start with the old way (so interns feel the problem), then introduce machine learning (the fix), then scale it on HPC, then make it usable. Every tool below is free and open-source except the commercial baseline.

Layer	Tool	Why / what interns learn
Notebooks	Google Colab (free GPU)	Zero-install Python + GPU for weeks 1–2; lowers the barrier for beginners.

Layer	Tool	Why / what interns learn
Language / data	Python, NumPy, scikit-image, matplotlib	Images as arrays of numbers; the foundation of everything.
Classic CV baseline	ImageJ / Fiji + scikit-image	Reproduce the commercial tool's thresholding & watershed — and watch it fail.
Deep learning	PyTorch	Tensors, autograd, the training loop — Nathan's specialty and the project's core.
Segmentation models	Cellpose-SAM & CellSAM (2025), U-Net	Two SAM-based generalist foundation models robust to weak boundaries, noise, and blur (both have no-code web apps for a fast first try); compare them, then fine-tune.
Annotation	Cellpose GUI / napari / QuPath	Human-in-the-loop labeling to make ground truth and improve the model.
Tracking	trackpy / btrack	Linking cells across frames to measure migration, division, proliferation.
HPC	SDSC Expanse (NVIDIA V100 GPUs) via NSF ACCESS; Slurm; JupyterHub	Login vs compute nodes, modules, batch jobs — real research computing.
Packaging	Git/GitHub, a one-command script, README	Reproducibility; turning a notebook into a tool others can run.
Baseline (compare)	The current commercial cell-analysis software	The accuracy/speed/access bar we are trying to beat.

4. The four-week arc at a glance

Each week the team ends with a working artifact and a one-slide share-out — so progress is visible and the final poster assembles itself from weekly pieces.

Week	Theme	By Friday the team has...
1	From microscope to numbers — Python, images as data, and why the old tool struggles	A working classic-CV baseline + a small hand-labeled test set + the failure cases documented.
2	Teaching a computer to see cells — deep learning segmentation	A PyTorch / Cellpose-SAM segmenter that beats the baseline on weak-boundary cells, with real accuracy numbers.
3	Scaling up & tracking over time — HPC + time-lapse	The pipeline running on SDSC, processing a full time-series with cell tracking and debris/scratch rejection.
4	Make it usable & tell the story — packaging, validation, poster	A one-command tool, a validation study vs the commercial software, and a finished poster + talk.

5. Daily schedule template

A humane full day (~5.5 hours of contact time). Adjust to OPALS hours. Mornings teach a concept; afternoons build the real project; the day ends with a short share-out and notebook entry.

Time	Block	Purpose
9:00	Stand-up (15 min)	Each intern: yesterday / today / one blocker. Builds the habit of a research team.
9:15	Concept lesson (75 min)	Mentor-led mini-lecture + live demo on the day's idea.
10:30	Break	—
10:45	Guided lab (90 min)	Everyone follows a notebook together; pairs help each other.
12:15	Lunch	—
1:00	Project build (2 hrs)	Leads work on their component of the shared tool; mentor circulates.
3:00	Break	—
3:15	Share-out + notebook (30 min)	One pair demos; everyone writes a notebook entry. Friday = weekly mini-talk.

6. Week 1 — From microscope to numbers

Goal: Python and image fluency, understand the biology and the problem, build the classic-CV baseline, and feel why it fails — motivating machine learning.

Day 1 Welcome & the problem

Concept (AM): Meet the team and the lab; tour of live-cell imaging; demo the commercial software and its pain points (tuning, slow, single license). Set up Google/Colab + GitHub accounts. Python crash course part 1: variables, lists, loops, functions.

Build (PM): Get Colab running; clone the team repo; first commit. Beginners: guided Python notebook. Stretch: write a function that reports image file info for the dataset.

Leads: All **Artifact:** Working Colab + GitHub for everyone

Day 2 Images are numbers

Concept (AM): Pixels, grayscale vs RGB, arrays, histograms. Biology mini-lecture: what fibroblasts are; what confluence, migration, and proliferation assays measure; why segmentation gives the numbers.

Build (PM): Load the .tif in Python (NumPy + scikit-image), display it, plot its histogram, find the scale bar, debris, and the scratch line. Stretch: crop tiles and compare brightness across the well.

Leads: Data **Artifact:** Notebook that loads & visualizes the real data

Day 3 The old way: classic computer vision

Concept (AM): How thresholding + watershed + “analyze particles” work (the commercial tool's approach). Where rule-based CV breaks down.

Build (PM): Reproduce it in Fiji/ImageJ, then in scikit-image (Otsu threshold, morphology, watershed). Save outputs. Stretch: tune parameters and log how results change.

Leads: Model **Artifact:** A classic-CV baseline segmentation

Day 4 What does “good” look like?

Concept (AM): Ground truth, annotation, and why we need it. Intro to evaluation: counting error and Intersection-over-Union (IoU) explained visually.

Build (PM): Hand-label cells in a few crops to make a small test set; write a script to compare baseline vs labels. Document failures: merged weak-boundary cells, scratch counted as a cell.

Leads: Data, Viz **Artifact:** Small ground-truth set + first metrics

Day 5 Why machine learning? (Week-1 mini-talks)

Concept (AM): Recap baseline limits with numbers. Conceptual intro to neural networks — learning patterns from examples instead of hand-written rules (no heavy math).

Build (PM): Each intern gives a 3-minute mini-talk on their week-1 piece (rehearsal for the symposium). Plan week 2 and confirm roles.

Leads: All **Artifact:** Week-1 mini-talks + project plan

7. Week 2 — Teaching a computer to see cells

Goal: understand supervised learning and build a PyTorch / Cellpose-SAM segmenter that beats the baseline on weak-boundary cells, with real accuracy numbers.

Day 6 First contact with a real model

Concept (AM): Supervised learning: training data, labels, loss, train/val/test, overfitting — told as a story.

Build (PM): Run two generalist foundation models — Cellpose-SAM and CellSAM — out-of-the-box on our fibroblasts and compare both to the day-3 baseline. The “wow” moment: they outline weak-boundary cells with no tuning. Decide which to fine-tune.

Leads: Model **Artifact:** Zero-shot segmentation from two models

Day 7 PyTorch fundamentals

Concept (AM): Tensors, autograd, and the training loop. Why GPUs make this fast (sets up the need for SDSC).

Build (PM): Train a tiny PyTorch classifier on a toy dataset; watch the loss go down. Stretch: change the network and see what happens to accuracy.

Leads: Model, HPC **Artifact:** A model the team trained themselves

Day 8 How segmentation models work

Concept (AM): Semantic vs instance segmentation; the U-Net architecture, conceptually. Data augmentation — rotations, noise, blur — and why it makes models robust to weak boundaries.

Build (PM): Assemble a training set (our labels + public cell datasets). Set up fine-tuning of Cellpose / a U-Net on it. Stretch: add augmentations and compare.

Leads: Data, Model **Artifact:** A curated training dataset + training run

Day 9 Human-in-the-loop labeling

Concept (AM): How a few good labels plus correction beats thousands of bad ones; active learning idea.

Build (PM): Use the Cellpose GUI / napari to label faster, correct the model's mistakes, and re-train. Watch accuracy climb. Stretch: target the model's worst images first.

Leads: Data **Artifact:** Improved, fine-tuned model

Day 10 Scoring the model (Week-2 mini-talks)

Concept (AM): Honest evaluation: IoU, F1, count accuracy; comparing model vs baseline vs ground truth.

Build (PM): Produce the comparison figures and a results table. Mid-program check-in; draft the poster outline. Week-2 mini-talks.

Leads: Viz, Model **Artifact:** Accuracy table + comparison figures

8. Week 3 — Scaling up & tracking over time

Goal: get onto SDSC, run at scale via Slurm, process a full time-lapse with tracking, and reject scratches/debris — measuring the speed win over the commercial tool.

Day 11 Meet the supercomputer

Concept (AM): What an HPC cluster is: login vs compute nodes, the file system, software modules, the Slurm scheduler, GPUs. The ACCESS / SDSC Expanse picture.

Build (PM): Log in to SDSC (Expanse / JupyterHub), move data over, run a first “hello GPU” job. Stretch: explore module load and nvidia-smi.

Leads: HPC **Artifact:** Everyone has run something on SDSC

Day 12 Batch processing at scale

Concept (AM): Writing a Slurm batch script and a job array; queueing vs interactive work; thinking about throughput.

Build (PM): Run the segmenter over many frames on Expanse with a job array. Time it and compare to the single licensed machine (the “it takes forever” pain point).

Leads: HPC, Model **Artifact:** Whole experiment segmented on HPC + speed numbers

Day 13 Tracking cells through time

Concept (AM): Linking objects across frames (centroid / IoU tracking); what migration, proliferation, and division look like in data.

Build (PM): Add tracking (trackpy / btrack) to the 69-hour series; produce per-cell tracks and population curves — the biology readout. Stretch: flag dividing cells.

Leads: Model, Viz **Artifact:** Tracks + migration/growth curves

Day 14 Rejecting scratches & debris

Concept (AM): Why the old tool counts scratches; using size/shape/texture features (or a tiny classifier) to filter non-cells; robustness via augmentation.

Build (PM): Add a debris/scratch filter; show the scratch line is no longer counted. Test on a hard well. Stretch: quantify how many false positives were removed.

Leads: Model, Data **Artifact:** Robust pipeline that ignores scratches

Day 15 Full pipeline on real data (Week-3 mini-talks)

Concept (AM): Putting it together: load → segment → track → measure, end-to-end on HPC.

Build (PM): Run the full pipeline on a complete experiment. Build the benchmark table: accuracy + speed + accessibility vs the commercial tool. Poster draft v1. Week-3 mini-talks.

Leads: All **Artifact:** End-to-end pipeline + benchmark table

9. Week 4 — Make it usable & tell the story

Goal: turn the pipeline into a one-command tool, validate it honestly against the commercial software, and create + rehearse the poster and talk.

Day 16 Make it one command

Concept (AM): From notebook to tool: a single script/config that takes a folder of images and outputs results + figures. Reproducibility (requirements file, README, fixed settings).

Build (PM): Wrap the pipeline; test it on a brand-new folder of images to prove “import → run → results.” Stretch: a simple config file for non-coders.

Leads: HPC, Model **Artifact:** One-command, import-and-run tool

Day 17 Validation study

Concept (AM): How scientists prove a method works: agreement, where each method wins, failure cases, honest limitations. Gentle stats (correlation, agreement).

Build (PM): Compare our tool vs the commercial software across multiple wells/timepoints; make agreement plots; write down limitations.

Leads: Data, Viz **Artifact:** Validation results + limitations

Day 18 Visualization & results

Concept (AM): What makes a scientific figure clear and honest; color, scale bars, captions; overlay videos.

Build (PM): Produce polished figures, a tracked-cell overlay video, and a simple results dashboard/HTML. Each lead contributes their component's figure.

Leads: Viz, All **Artifact:** Final figures, video, dashboard

Day 19 Build the poster & talk

Concept (AM): Story structure: problem → approach → results → impact (accurate, fast, free, accessible). Designing a poster; giving a short talk.

Build (PM): Assemble the poster and slides from weekly pieces; each intern drafts their section. First rehearsal with feedback.

Leads: All **Artifact:** Poster v2 + talk slides

Day 20 Symposium

Concept (AM): Dress rehearsal; timing; handling questions.

Build (PM): Present talk/poster at the OPALS symposium. Hand off the GitHub repo + README so the lab can keep using it. Reflection and feedback.

Leads: All **Artifact:** Final presentation + delivered tool

10. Optional follow-up — weeks 5–8

For interns who continue (e.g. into the spring track or an independent project), the tool becomes a real research and publication effort.

Week	Focus	What gets built
5	Generalize	Apply the tool to other cell types / other labs' images; fine-tune for broader coverage; active-learning to label efficiently.
6	Richer tracking & analytics	Detect cell division and build lineage trees; full migration/proliferation analytics that match or exceed the commercial metrics.
7	Productionize	A simple web UI (Gradio/Streamlit) or napari plugin so non-coders in the lab can use it; documentation; one-click inference.
8	Write it up	A short preprint / Journal of Emerging Investigators submission or science-fair (ISEF) entry; present to the PI; open-source release.

11. Skills & learning outcomes

Mapped to the weeks where each is introduced and practiced.

Skill	W1	W2	W3	W4
Python, NumPy, image processing	Intro	Use	Use	Use
Git / GitHub & reproducibility	Intro	Use	Use	Master

Skill	W1	W2	W3	W4
ML concepts (data, loss, overfitting)	—	Intro	Use	Use
PyTorch & training loops	—	Intro	Use	Use
Segmentation models (U-Net, Cellpose-SAM)	—	Intro	Use	Use
Evaluation metrics (IoU, F1)	Intro	Use	Use	Master
HPC: Slurm, GPUs, batch jobs (SDSC)	—	—	Intro	Use
Cell tracking & quantitative biology	—	—	Intro	Use
Scientific visualization & figures	Intro	Use	Use	Master
Science communication (poster/talk)	Intro	Use	Use	Master

12. Assessment & milestones

Assessment is project-based and visible, not test-based:

- **Weekly artifact + mini-talk:** each Friday the team demos a working result and each intern speaks for ~3 minutes.
- **Daily commits & notebook:** steady participation in the shared repo and a kept research notebook.
- **Final deliverables:** (1) the working one-command tool in GitHub, (2) the validation study vs the commercial software, (3) the OPALS poster and talk.

13. Mentor prep checklist (before Day 1)

A few items have real lead time — start the starred ones several weeks out.

1. ★ **ACCESS allocation:** request an NSF ACCESS allocation (the “Explore” tier is sized for education/training) for SDSC Expanse GPU well in advance; it is not instant.
2. ★ **Minor-access policy:** confirm with SDSC how interns under 18 can access the system. If individual accounts aren't possible in time, fall back to mentor-supervised runs on a shared training allocation, or do weeks 1–3 on Colab/Colab Pro or a lab GPU and treat SDSC as the scale-up demo.
3. **Dataset:** organize a representative set of .tif images (multiple wells/timepoints) and pre-curate a small hand-labeled subset so Week 1 isn't blocked.
4. **Baseline outputs:** export the current commercial software's results on the same images for the head-to-head comparison.
5. **Accounts & repo:** create a GitHub org/repo with a starter README and Colab notebook templates; prepare Google accounts/Colab access.
6. **Environment:** pre-test Cellpose-SAM, PyTorch, and the tracking libraries on both Colab and SDSC so Day 1 isn't an install marathon.

7. **Logistics:** room, monitors/laptops, lab-safety orientation, and confirm the symposium date and poster format.

14. Risks & contingencies

Risk	Mitigation
HPC access slips (allocation or minor-access delay)	Run weeks 1–2 entirely on Colab; demo SDSC in week 3 via a shared/mentor account; the project still fully works on a single GPU.
Annotation is too slow	Lean on Cellpose-SAM zero-shot + light human correction instead of labeling from scratch.
Wide skill gap stalls the group	Pair programming, core+stretch tasks, and clearly scoped roles keep everyone contributing at their level.
A week runs long	Each week's Friday is a buffer/integration day; cut stretch goals before core goals.
Model underperforms on our hardest images	That's a finding, not a failure — document it as a limitation and a future-work item in the poster.

15. Key resources

- OPALS / UCSD IEM: iem.ucsd.edu/programs/opals
- SDSC Expanse user guide: sdsc.edu/systems/expanse
- ACCESS allocations: allocations.access-ci.org
- Cellpose (incl. Cellpose-SAM): github.com/MouseLand/cellpose
- Cellpose-SAM paper (2025): biorxiv.org/content/10.1101/2025.04.28.651001v1
- CellSAM (code + web app): github.com/vanvalenlab/cellSAM · cellsam.deepcell.org
- PyTorch tutorials: pytorch.org/tutorials